LINQ - Complete Practice Question Set

Includes complete single-table and multiple-table sample data for Employee, Department, and Project, followed by practical LINQ exercises from basic filtering to interview-level EF Core thinking.

1. Models

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int DepartmentId { get; set; }
    public decimal Salary { get; set; }
    public bool IsActive { get; set; }
}

public class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
}

public class Project
{
    public int Id { get; set; }
    public string ProjectName { get; set; }
    public int EmployeeId { get; set; }
}
```

2. Employee Data

```
var employees = new List<Employee>
{
    new Employee { Id = 1, Name = "Rohit", DepartmentId = 1,
        Salary = 60000, IsActive = true },

    new Employee { Id = 2, Name = "Amit", DepartmentId = 2,
        Salary = 45000, IsActive = true },

    new Employee { Id = 3, Name = "Pavan", DepartmentId = 1,
        Salary = 80000, IsActive = false },

    new Employee { Id = 4, Name = "Raj", DepartmentId = 1,
        Salary = 70000, IsActive = true },

    new Employee { Id = 5, Name = "Neha", DepartmentId = 2,
        Salary = 55000, IsActive = true },

    new Employee { Id = 6, Name = "Karan", DepartmentId = 3,
        Salary = 65000, IsActive = true }
};
```

3. Department Data

```
var departments = new List<Department>
{
    new Department { Id = 1, Name = "IT" },
    new Department { Id = 2, Name = "HR" },
    new Department { Id = 3, Name = "Finance" },
    new Department { Id = 4, Name = "Admin" }
};
```

4. Project Data

```
var projects = new List<Project>
{
    new Project { Id = 101, ProjectName = "Inspection Service",
        EmployeeId = 1 },

    new Project { Id = 102, ProjectName = "RLI API",
        EmployeeId = 1 },
```

```
    new Project { Id = 103, ProjectName = "Audit Service",
        EmployeeId = 3 },

    new Project { Id = 104, ProjectName = "Planck Integration",
        EmployeeId = 4 },

    new Project { Id = 105, ProjectName = "HR Portal",
        EmployeeId = 2 }
};
```

Relationship Map

```
Employee.DepartmentId  --->  Department.Id

Project.EmployeeId       --->  Employee.Id

So:
Employee belongs to one Department.
Employee can have zero, one, or many Projects.
Department can have zero, one, or many Employees.

Useful test cases already included:
- Rohit has 2 projects.
- Neha has no project.
- Karan has no project.
- Admin department has no employees.
- Pavan is inactive.
```

Practice Questions

Level 1 - Filtering

1. Get all active employees.

2. Get employees whose salary is greater than 50,000.

3. Get all employees from the HR department.

4. Get employees who are active AND have salary greater than 50,000.

5. Get employees who belong to IT OR HR.

6. Get employees whose name starts with R.

7. Get employees whose name contains the letter a.

Level 2 - Finding, Checking & Counting

8. Find the employee whose Id is 3.

9. Find the first IT employee.

10. Check whether any inactive employee exists.

11. Check whether any employee earns more than 75,000.

12. Check whether all employees are active.

13. Count active employees.

14. Count IT employees.

15. Check whether Finance has at least one active employee.

Level 3 - Projection & Sorting

16. Sort employees by salary lowest to highest.

17. Sort employees by salary highest to lowest.

18. Sort employees alphabetically by Name.

19. Get only employee names.

20. Return only Name and Salary for active employees.

21. Get the top 2 highest-paid employees.

22. Get the second-highest-paid employee.

23. Sort by DepartmentId and then Salary descending.

24. Return Id, Name and Salary for employees earning above 50,000.

Level 4 - Aggregation

25. Find total salary of all employees.

26. Find average salary.

27. Find highest salary.

28. Find lowest salary.

29. Find total salary of IT employees.

30. Find average salary of active employees.

31. Count employees earning above the overall average salary.

32. Calculate the difference between highest and lowest salary.

Level 5 - GroupBy

33. Group employees by DepartmentId.

34. Return employee count for each department.

35. Calculate total salary department-wise.

36. Calculate average salary for each department.

37. Return DepartmentId, EmployeeCount, TotalSalary, AverageSalary and HighestSalary.

38. Find department with highest total salary.

39. Find department with highest average salary.

40. Find highest-paid employee in each department.

Level 6 - Practical Single-Collection Problems

41. Get active employees sorted by salary descending and return Id, Name and Salary.

42. Find highest-paid active IT employee.

43. Get employees earning above company average salary.

44. Add duplicate names and find duplicate employee names.

45. Get distinct DepartmentIds.

46. Implement PageNumber=2 and PageSize=2 using Skip and Take.

47. Build optional filtering with department, minimumSalary and isActive; apply only provided filters.

Level 7 - Employee + Department

48. Join Employee and Department; return EmployeeName and DepartmentName.

49. Return active IT employees using DepartmentId relationship.

50. Return EmployeeName, DepartmentName and Salary sorted descending.

51. Find HR employees using a join.

52. Find highest-paid employee in each department.

53. Return DepartmentName, EmployeeCount, TotalSalary and AverageSalary.

54. Return all departments even if they have zero employees.

55. Find departments with no employees.

Level 8 - Employee + Project

56. Join Employee and Project; return EmployeeName and ProjectName.

57. Find all projects assigned to Rohit.

58. Find employees working on more than one project.

59. Return EmployeeName and ProjectCount for employees with projects.

60. Find highest-paid employee assigned to a project.

61. Find employees with multiple projects and salary above 50,000.

62. Find employees with no project.

63. Return all employees including those without projects; show No Project when missing.

Level 9 - Three-Table LINQ

64. Join Employee, Department and Project; return EmployeeName, DepartmentName, ProjectName and Salary.

65. Find active IT employees assigned to at least one project.

66. Find employees earning above 50,000 with department and project information.

67. Find department whose employees work on the most projects.

68. Return DepartmentName, EmployeeCount, ActiveEmployeeCount, TotalSalary and ProjectCount for every department.

69. Find employees earning more than their own department average.

70. Return top-paid employee from each department with department name.

71. Find departments with no active employees.

Level 10 - Interview / Production Challenges

72. Build a nested response: Department -> Employees -> Project names.

73. Find departments whose average salary is above company-wide average.

74. Find employees sharing the same salary.

75. Return top 2 highest-paid employees from each department.

76. Return employees who have no project OR are inactive, with department name.

77. Dynamically compose filters for DepartmentId, MinimumSalary, MaximumSalary, IsActive and SearchText.

78. Convert a multi-table exercise to EF Core and explain what SQL you expect before materialization.

79. Move ToListAsync() too early in an EF Core query, explain the performance impact, then fix it.

80. Implement the same related-data result once using navigation-property projection and once using explicit Join; explain when each is preferable.

81. Design a service method supporting filtering, sorting and pagination while projecting only required DTO columns.

How to Practice

1. Attempt each question yourself before checking syntax.

2. Keep the data inside your service and return the result through your existing API/controller.

3. Use breakpoints and the Visual Studio Locals/Watch windows to inspect the LINQ result.

4. For multiple-table questions, first draw the relationship using the foreign keys.

5. After the List version works, repeat the important questions using EF Core DbSet.

6. Pay close attention to where ToList()/ToListAsync() is called, especially when you later work with IQueryable.

7. For interviews, explain your LINQ in plain English as well as writing the syntax.